Feature Matching for SLAM

Sami Osborn

February 2026

Contents

1 Notation, Conventions, and the Matching Problem **2**

1.1 Keypoints, patches, and descriptors 2

1.2 Matches as index pairs 2

2 Patch Similarity via Normalised Cross-Correlation (NCC) **3**

2.1 Vectorised patches 3

2.2 Normalisation: z-score and L2 3

2.3 NCC as cosine similarity 3

3 Descriptor Similarity: L2 and Hamming **3**

3.1 Float descriptors and L2 distance 3

3.2 Binary descriptors and Hamming distance 4

4 Matching Policies: NN, Mutual Check, Ratio Test, and Top-K **4**

4.1 Nearest neighbour matching 4

4.2 Mutual cross-check 4

4.3 Ratio test 4

4.4 Top-K filtering 5

5 Geometric Verification with a Homography and RANSAC **5**

5.1 When a homography is the right model 5

5.2 Homography mapping and transfer error 5

5.3 RANSAC 5

6 Visual Sanity Checks **6**

6.1 Match visualisation 6

6.2 Projecting source corners into the scene 6

7 Rotation and Scale Robustness **6**

7.1 Keypoint orientation 6

7.2 Gaussian pyramids and scale 6

7.3 Multiscale BRIEF and scale gating 6

8 Practical Tuning and Debugging **7**

8.1 Parameter tradeoffs 7

1 Notation, Conventions, and the Matching Problem

1.1 Keypoints, patches, and descriptors

Definition 1.1 (Keypoints)**.** Let $I : \Omega \subset \mathbb{Z}^2 \to \mathbb{R}$ be a grayscale image. A set of detected keypoints is a finite collection

$$\mathcal{K} := \{x_i\}_{i=1}^{N}, \qquad x_i \in \Omega.$$

In implementations, we store keypoints as an array of shape $(N, 2)$ with (u, v) or (x, y) coordinates.

Definition 1.2 (Patch / window around a keypoint)**.** Let $\mathcal{W} \subset \mathbb{Z}^2$ be a finite symmetric set (e.g. $\{-r, \dots, r\}^2$). The patch centred at x is

$$P_I(x) \;:=\; \{I(x + r) : r \in \mathcal{W}\}.$$

For square patches of odd size $P = 2r + 1$, we often identify $P_I(x)$ with a matrix in $\mathbb{R}^{P\times P}$.

Definition 1.3 (Descriptor)**.** A descriptor is a map that assigns to each keypoint x a vector (or bitstring) summarising local appearance:

$$d(x) \in \mathbb{R}^D \quad \text{(float descriptor)} \qquad \text{or} \qquad d(x) \in \{0, 1\}^B \quad \text{(binary descriptor)}.$$

The descriptor may be computed from a patch, gradients, or other local statistics.

Remark 1.4. In our codebase we used two families: (i) *patch similarity* via NCC on raw patches, and (ii) *descriptor similarity* via L2 distance for float descriptors and Hamming distance for binary BRIEF-like descriptors.

1.2 Matches as index pairs

Definition 1.5 (Match set)**.** Let $\mathcal{K}_0 = \{x_i\}_{i=1}^{N_0}$ be keypoints in image I_0 and $\mathcal{K}_1 = \{y_j\}_{j=1}^{N_1}$ in image I_1. A set of matches is a set of index pairs

$$\mathcal{M} \subset \{1, \dots, N_0\} \times \{1, \dots, N_1\}.$$

A match $(i, j) \in \mathcal{M}$ asserts that x_i and y_j correspond to (approximately) the same physical scene point.

Definition 1.6 (Score convention)**.** A match may carry a score s_{ij} where "better" depends on the similarity measure:

$$\text{NCC: higher is better (closer to 1)}, \qquad \text{Distances (L2/Hamming): lower is better.}$$

In code it is convenient to convert distances to a "higher is better" score via $s = -d$.

Remark 1.7. This score convention matters for sorting and for top-K filtering prior to RANSAC.

2 Patch Similarity via Normalised Cross-Correlation (NCC)

2.1 Vectorised patches

Definition 2.1 (Vectorised patch)**.** Given a patch $P \in \mathbb{R}^{P \times P}$, define its vectorisation $p \in \mathbb{R}^D$ with $D = P^2$ by stacking entries.

2.2 Normalisation: z-score and L2

Definition 2.2 (Per-patch z-score)**.** For a vectorised patch $p \in \mathbb{R}^D$, define

$$\tilde{p} \;:=\; \frac{p - \mu(p)\mathbf{1}}{\sigma(p) + \varepsilon},$$

where $\mu(p) = \frac{1}{D}\sum_{k=1}^{D} p_k$ and $\sigma(p)$ is the standard deviation.

Definition 2.3 (Row-wise L2 normalisation)**.** For $z \in \mathbb{R}^D$, define

$$\hat{z} \;:=\; \frac{z}{\|z\|_2 + \varepsilon}.$$

Remark 2.4. In our NCC implementation, we typically use both: first z-score to remove brightness/contrast offsets within the patch, then L2-normalise so a dot product becomes a cosine similarity.

2.3 NCC as cosine similarity

Proposition 2.5 (NCC as dot product of normalised patches)**.** *Let $p, q \in \mathbb{R}^D$ be two vectorised patches. If $\hat{p}, \hat{q}$ are z-scored and L2-normalised versions, then*

$$\mathrm{NCC}(p, q) \;\approx\; \hat{p}^{\mathsf{T}}\hat{q} \in [-1, 1].$$

Proof. Z-scoring enforces approximate zero mean and unit variance, while L2-normalisation enforces unit norm. The dot product of unit vectors is the cosine of the angle between them, bounded in $[-1, 1]$. For standard NCC definitions, this matches the normalised correlation coefficient up to finite-sample conventions and the stabilising ε. □

Remark 2.6. NCC is simple and often works surprisingly well for small viewpoint changes, but it is not scale-invariant and struggles under significant rotation unless the patch is canonicalised (e.g. by orientation) or matched in a pyramid.

3 Descriptor Similarity: L2 and Hamming

3.1 Float descriptors and L2 distance

Definition 3.1 (L2 distance)**.** For float descriptors $a, b \in \mathbb{R}^D$, define

$$d_{L2}(a, b) \;:=\; \|a - b\|_2.$$

Remark 3.2. SIFT-style pipelines often use L2 distance on gradient-histogram descriptors. In our minimal codebase, we primarily used NCC on raw patches, but the L2 machinery is the same pattern: compute a distance matrix, take nearest neighbours, then apply a filtering policy.

3.2 Binary descriptors and Hamming distance

Definition 3.3 (Binary descriptor)**.** A binary descriptor is a bitstring $b \in \{0,1\}^B$. In code, we often pack bits into bytes (uint8) for fast XOR operations.

Definition 3.4 (Hamming distance)**.** For two binary descriptors $b, c \in \{0,1\}^B$, the Hamming distance is

$$d_H(b,c) \;:=\; \sum_{k=1}^{B} \mathbf{1}\{b_k \neq c_k\}.$$

Equivalently, d_H is the popcount of $(b \oplus c)$.

Remark 3.5. Hamming distance is cheap to compute and robust to mild noise. It is widely used in ORB/BRIEF-style pipelines.

4 Matching Policies: NN, Mutual Check, Ratio Test, and Top-K

4.1 Nearest neighbour matching

Definition 4.1 (Nearest neighbour (NN) matching)**.** Given descriptors $\{d_i\}_{i=1}^{N_0}$ in I_0 and $\{e_j\}_{j=1}^{N_1}$ in I_1, define for each i

$$j^\star(i) \in \arg\min_j \; d(d_i, e_j),$$

where $d(\cdot,\cdot)$ is a distance (L2 or Hamming). NN matching returns candidate pairs $(i, j^\star(i))$.

Remark 4.2. For NCC, we instead *maximise* similarity. The pattern is the same: choose best partner per descriptor/patch under the chosen score.

4.2 Mutual cross-check

Definition 4.3 (Mutual (symmetric) matching)**.** A match (i,j) is *mutual* if $j = j^\star(i)$ and also $i = i^\star(j)$, where

$$i^\star(j) \in \arg\min_i d(d_i, e_j).$$

Remark 4.4. Mutual matching reduces false matches but can remove many true matches when the descriptor space is ambiguous or when one side contains repeated texture.

4.3 Ratio test

Definition 4.5 (Lowe-style ratio test)**.** Let $j_1(i)$ and $j_2(i)$ be the nearest and second-nearest neighbours of d_i by distance. The ratio test accepts i if

$$\frac{d(d_i, e_{j_1(i)})}{d(d_i, e_{j_2(i)})} < \rho,$$

for a threshold $\rho \in (0,1)$.

Remark 4.6. For Hamming distances (binary descriptors), the ratio test is often *too strict* early on unless the scene has very distinctive texture. NN + max-distance threshold tends to be the more forgiving baseline.

4.4 Top-K filtering

Definition 4.7 (Top-K pre-filtering)**.** Given a set of candidate matches with scores $\{s_m\}_{m=1}^{M}$ (higher is better), *top-K* filtering keeps only the K highest-scoring matches prior to geometry verification (e.g. RANSAC).

Remark 4.8. Top-K is useful when you have many weak matches: it biases RANSAC toward sampling higher-quality correspondences.

5 Geometric Verification with a Homography and RANSAC

5.1 When a homography is the right model

Remark 5.1 (Planar scenes and "box on a plane")*.* A homography is appropriate when the matched points lie on a plane in 3D or when the camera rotates about its optical centre with negligible translation. For a printed box face or planar object, homography verification is a good sanity check.

5.2 Homography mapping and transfer error

Definition 5.2 (Homography mapping)**.** A homography is a projective transform $H \in \mathbb{R}^{3\times 3}$ acting on homogeneous image points:

$$\tilde{y} \;\sim\; H\tilde{x},$$

where $\tilde{x} = (u, v, 1)^{\mathsf{T}}$ and $\sim$ denotes equality up to nonzero scale.

Definition 5.3 (Forward transfer error)**.** Given corresponding inhomogeneous points $x_i, y_i \in \mathbb{R}^2$, define

$$e_i(H) \;:=\; \|y_i - \pi(H\tilde{x}_i)\|_2,$$

where $\pi(\cdot)$ dehomogenises homogeneous coordinates.

Definition 5.4 (Symmetric transfer error)**.** Define the symmetric transfer error by

$$e_i^{\mathrm{sym}}(H) \;:=\; \|y_i - \pi(H\tilde{x}_i)\|_2^2 \;+\; \|x_i - \pi(H^{-1}\tilde{y}_i)\|_2^2.$$

Remark 5.5. Symmetric transfer error penalises mismatch in both directions and is commonly used for robust homography scoring.

5.3 RANSAC

Definition 5.6 (RANSAC for homography)**.** Given candidate correspondences $\{(x_i, y_i)\}_{i=1}^{M}$:

1. Repeat for T trials: sample 4 correspondences uniformly without replacement.
2. Fit a homography H to the sample (e.g. DLT + normalisation).
3. Score H by counting inliers under a threshold τ using $e_i^{\mathrm{sym}}(H)$.
4. Keep the H with the largest inlier count (break ties by lower mean inlier error).

Return the best H and inlier mask.

Remark 5.7. RANSAC is the key "outlier rejection" step: it lets you tolerate many incorrect matches, as long as a consistent subset exists.

6 Visual Sanity Checks

6.1 Match visualisation

Remark 6.1. A match plot (two images side-by-side with connecting lines) is the fastest sanity check. If the majority of lines criss-cross wildly, your matching policy is too permissive or your descriptor is not robust for the transform at hand.

6.2 Projecting source corners into the scene

Definition 6.2 (Projected box corners)**.** Let I_0 be the template image (e.g. `box.png`). Its four corners define points $x_1, \ldots, x_4 \in \mathbb{R}^2$. Using an estimated homography H, the projected corners in I_1 are

$$y_k \;\coloneqq\; \pi(H\tilde{x}_k), \qquad k = 1, \ldots, 4.$$

Drawing the polygon through $\{y_k\}$ on I_1 provides a geometric sanity check.

Remark 6.3. If the polygon is wildly distorted or misplaced, your estimated H is likely being fit to the wrong inlier set (or the model is inappropriate).

7 Rotation and Scale Robustness

7.1 Keypoint orientation

Definition 7.1 (Keypoint orientation (local gradient angle))**.** Given a keypoint x, define a local orientation $\theta(x)$ from a neighbourhood around x (e.g. a weighted sum of gradients):

$$\theta(x) \;\coloneqq\; \operatorname{atan2}\Big(\sum_{r\in\mathcal{W}} w(r)\,\partial_v I(x+r),\; \sum_{r\in\mathcal{W}} w(r)\,\partial_u I(x+r)\Big).$$

Remark 7.2. Orientation is used to "rotate" the descriptor sampling pattern so that the descriptor becomes approximately rotation-invariant.

7.2 Gaussian pyramids and scale

Definition 7.3 (Gaussian pyramid (recall))**.** Define $I^{(0)} = I$ and

$$I^{(\ell+1)} \coloneqq \big(G_\sigma * I^{(\ell)}\big)^{\downarrow 2}.$$

Each level ℓ corresponds to a larger effective spatial scale in the original image.

Remark 7.4. If an object appears smaller in the scene image than in the template image, matching descriptors across pyramid levels can recover correspondences that a single-scale descriptor misses.

7.3 Multiscale BRIEF and scale gating

Definition 7.5 (Multiscale BRIEF (concept))**.** Compute BRIEF descriptors for keypoints at multiple pyramid levels. A match may be constrained to plausible relative scale levels (a *scale gate*) to reduce spurious matches across very different scales.

Remark 7.6. This is the core idea behind scale robustness: instead of trying to make a single descriptor perfectly scale-invariant, you search across a discrete set of scales.

8 Practical Tuning and Debugging

8.1 Parameter tradeoffs

Remark 8.1. Typical tradeoffs observed in practice:

- Larger patch size increases distinctiveness but may reduce repeatability under viewpoint changes.
- More BRIEF bits increases distinctiveness but may require relaxing the Hamming threshold to avoid zero matches.
- Mutual cross-check reduces false matches but often reduces inliers too much early on.
- RANSAC threshold τ should reflect expected pixel localisation noise and model mismatch.